

Langage à Objets Avancé

Projet N° 02

January 16, 2022

1 Introduction

Un circuit combinatoire est un ensemble de portes logiques reliées entre elles pour répondre à une expression algébrique. Dans le projet , nous allons implémenter un ensemble de types et de fonctions qui nous permettront de créer un circuit combinatoire et de l'afficher. En plus , on doit pouvoir voir la progression de l'information pas à pas à chaque fois qu'elle franchit une porte logique et d'afficher sous forme textuelle les fonctions de sortie à l'aide des variables d'entrée. En outre , le circuit peut etre généré à partir de son expression textuelle, sauvé dans un fichier pour le lire ensuite.

2 Les classes

2.1 La classe "Gate"

Nous avons défini une nouvelle entité Gate , qui est la classe mère de toutes les autres classes qui définissent les portes logiques. Elle est caractérisé par:

- string name : un nom(qui est sur trois caractères , par exemple pour une porte "And" ça sera "AND" , et pour une porte "Nand" , ça sera "NAN" etc..)
- vector <Gate * >* : En plus du nom, une Gate possède un vecteur d'entrées (On peut avoir plus de deux entrées) , ce vecteur (qui est un pointeur vers) un tableau qui contient des pointeurs vers des objets de type Gate
- bool valeurBooleenne : Une Gate possède une valeur de sortie qui sera calculée en fonction de la porte logique.

De la classe Gate héritent plusieurs classe qui sont :

1. AndGat : réalise AND(a1, a2, .. an) et retourne : a1 && a2 && .. && an
2. NandGate : réalise NAND(a1, a2, .. an) et retourne : ! (a1&& a2 && ..&& an)
3. OrGat : réalise OR(a1, a2, .. an) et retourne : a1 — a2 OR .. OR an
4. NorGate : réalise AND(a1, a2, .. an) et retourne : !(a1 OR a2 OR ..OR an)
5. NegateGat : réalise NEGATE(a) et retourne !a
6. XorGate : réalise Xor(a,b) et retourne : !ab OU !ba
7. XnorGat : réalise Xor(a,b) et retourne : ! (!ab OR !ba)*
8. InputGate : Une classe particulière des sous classes de Gate, elle possède un constructeur qui initialise le nom de l'input et sa valeur

9. OutputGate : Une classe particulière des sous classes de Gate, elle possède un constructeur qui initialise le nom de l'input et sa valeur

Les classes citées en dessus , redéfinissent la méthode `getValeurBooleenne()` de la classe mère, et calculent la sortie en fonction du type. La construction des objets , se passe en appelant le constructeur de la classe mère pour instancier une Gate avec le bon nom , puis l'initialisation du vecteur des entrées.

2.2 La classe "Circuit"

Cette classe représente le coeur du projet, et qui possèdent les méthodes permettant de répondre aux spécifications demandées . La création du circuit initialise le vecteur des entrées , le vecteur des portes logiques ainsi le vecteur de sortie. La classe "Circuit " possède comme données :

1. `vector <InputGate * >* inputs` : qui est le tableau contenant les entrées du circuit
2. `vector<OutputGate*>* outputs` : qui est le tableau de sorties du circuit (dans l'exemple il y'avais qu'une sortie mais il peut y'avoir plusieurs)
3. `vector <Gate*>* gates` : contient toutes les portes logiques qui constituent le circuit
4. `vector <vector <char>* >* affichageCircuit` : le tableau qui représente le circuit logique, qui est un tableau bidimensionnel de caractères : qui peuvent être des "-" ou des "*" ou des "+" ou bien des caractères alphabétique pour représenter les portes logiques"
5. `vector <vector <Gate*>* >* simulationPasParPas` : nous permet de faire la simulation en mode pas à pas : A chaque étape ,on garde le vecteur des gates qu'on a pour pouvoir voir la progression de l'information

La classe "Circuit" offre une interface aux autres classes en mettant les méthodes essentielles en publiques et en cachant les autres (nécessaires pour le fonctionnement des autres méthodes mais qui ne doivent pas nécessairement être vues par l'utilisateur) et qui sont :

1. `void afficheCircuit() const` : cette méthode permet d'afficher le circuit (représentation textuelle) que nous avons représenté sous forme d'un tableau bidimensionnel de caractères.
2. `void simulation()` Permet la simulation en mode pas à pas
3. `void changerValeursDesPortesEntree()` ça nous permet de Changer les valeurs des portes d'entrée
4. `Circuit* expressionTextuelleToCircuit(const string expressionTextuelle)` Construire un circuit à partir d'une représentation textuelle et le renvoyer
5. `void sauvegarderCircuitDansFichier(const string nomFichier) const` : Permet de sauvegarder un circuit dans un fichier ayant comme nom `nomFichier`
6. `string afficherSousFormeTextuelle() const` : permet d'afficher le circuit sous la représentation textuelle

Les autres méthodes sont mises en privé , et sont appelées lors de la création du circuit, faisant des traitements necessaire à l’affichage du circuit.

1. `vector <Gate*>* ajoutInputs()` :Une methode qui recherche les positions des caractères '+' et '*' dans la dernière ligne du vecteur qui stoke l’affichage du circuit et ajoute aux memes positions dans la ligne en dessous le caractère '—'
2. `void ajoutCheminsApresInputs()` : Créer le chemin près avoir rajouté un input ie: (mettre les '—' , '*', '+' dans le bon endroit
3. `void ajoutNomsOperationsLogiques(const vector <Gate*>* portesLogiquesAjouter-Affichage)` : mettre les bonnes portes logique au bon endroit : en mettant le nom représenté sur trois caractères
4. `void ajoutCheminsApresOperationsLogiques()`
5. `void ajoutAsterisquesApresChemins(unsigned int level)`
6. `vector <Gate*>* trouverLesPortesLogiquesSuivantes(const vector<Gate*>* portesLogiquesPrecedentes);`

2.3 La classe Main

C’est la classe principale dans laquelle on appelle les méthodes (ci dessus) correspondantes aux traitements qu’on souhaite effectuer, on a un menu de la forme :

- Afficher le circuit : appelle la méthode : `afficheCircuit()`
- Simulation en mode pas a pas, appelle la méthode : `simulation()`
- Changer les valeurs des portes d entree : `void changerValeursDesPortesEntree()`
- Afficher sous forme textuelle les fonctions de sortie a l’aide des variables d’entrée
- Synthétiser un circuit à partir d’une expression textuelle : `Circuit* expressionTextuelleToCircuit(const string expressionTextuelle)`
- Sauver un circuit dans un fichier
- Relire un circuit qui est dans un fichier

2.4 Code source!

Lien vers le code source du projet : [Projet CPP - Sujet 02 - Ait benali Faycal / Lenny lamounaru](#)

2.5 Exécution :